

Assignment 6. Constraint Satisfaction Problems (CSP)

Hwanjo Yu
CSED342 - Artificial Intelligence

Contact: Junyoung Kim (junyoungkim@postech.ac.kr)

Deadline: May 11, 2025, at 23:59 (50% penalty per day of delay)

General Instructions

Assignment 6 consists of three problems, including both written tasks  and programming tasks . For the written problems, please submit your answers and solutions in **PDF** format. You may create the PDF by converting from Overleaf or by taking clear, legible photos of your handwritten solutions and combining them into a single PDF; any issues arising from illegible handwriting are solely your responsibility.

For the programming problem, please note that the assignment has been developed in Python 3.12, so please use **Python 3.12** to implement your code. You must submit the programming file as **submission.py** without changing the file name.

Note : Please use only the libraries provided in the code. Do not import any additional libraries.

Ultimately, your final submission must be a `{YOUR_STUDENT_ID}.zip` file that contains both `{YOUR_STUDENT_ID}.pdf` (your written solution) and `submission.py` (your code).

Note : Submissions that do not follow the correct format will be subject to a **30% deduction**.

When working on your code, please make your modifications only within the designated section in `submission.py`

Your code will be evaluated on two types of test cases, basic and hidden, which you can see in `grader.py`. Basic tests, which are fully provided to you, do not stress your code with large inputs or tricky corner cases. Hidden tests are more complex and do stress your code. The inputs of hidden tests are provided in `grader.py`, but the correct outputs are not. To run the tests, you will need to have `graderUtil.py` in the same directory as your code and `grader.py`.

Then, you can run all the tests by typing

```
python grader.py
```

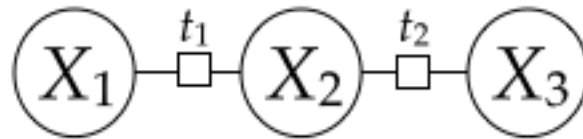
This will tell you only whether you passed the basic tests. On the hidden tests, the script will alert you if your code takes too long or crashes, but does not say whether you got the correct output. You can also run a single test (e.g., 3a-0-basic) by typing

```
python grader.py 3a-0-basic
```

We strongly encourage you to read and understand the test cases, create your own test cases, and not just blindly run `grader.py`.

Problem 1 [3 Points]

let's consider a simple CSP with 3 variables and 2 binary factors:



where $X_1, X_2, X_3 \in \{0, 1\}$ and t_1, t_2 are XOR functions (that is $t_1(X) = x_1 \oplus x_2$ and $t_2(X) = x_2 \oplus x_3$).

Problem 1a [1 Points]

Find the number of consistent assignments for this CSP. Please provide only the numeric value.

Problem 1b [3 Points]

Let's use backtracking search to solve the CSP. The `Backtrack()` algorithm as defined in the lectures is a recursive algorithm, where new instances of `Backtrack()` are called within parent instances of `Backtrack()`.

In this problem, we will ask you to produce the call stack for a specific call to `Backtrack()`. A call stack is just a diagram tracing out every recursive call. For our purposes, for each call to `Backtrack()` you should specify which variable is being assigned, the current domains, and which parent call to `Backtrack()` it's called within. For example, if the order in which we assign variables is X_1, X_2, X_3 , the call stack would be as follows:

$\{[01], [01], [01]\}$	$\xrightarrow{X_1=0}$	$\{\mathbf{0}, [01], [01]\}$	$\xrightarrow{X_2=1}$	$\{\mathbf{0}, \mathbf{1}, [01]\}$	$\xrightarrow{X_3=0}$	$\{\mathbf{0}, \mathbf{1}, \mathbf{0}\}$
------------------------	-----------------------	------------------------------	-----------------------	------------------------------------	-----------------------	--

$\xrightarrow{X_1=1}$	$\{\mathbf{1}, [01], [01]\}$	$\xrightarrow{X_2=0}$	$\{\mathbf{1}, \mathbf{0}, [01]\}$	$\xrightarrow{X_3=1}$	$\{\mathbf{1}, \mathbf{0}, \mathbf{1}\}$
-----------------------	------------------------------	-----------------------	------------------------------------	-----------------------	--

The notation $\{1, [01], [01]\}$ means that X_1 has been assigned value 1, while X_2 and X_3 are currently unassigned and each have domain $\{0, 1\}$. Note that we omit the comma in the domain for easier reading. We also avoid the weight variable for simplicity; the only possible weights for this problem are 0 and 1. In this case, backtrack is called 7 times. Notice that `Backtrack()` is not called when there's an inconsistent partial assignment ($\delta = 0$); for example, we don't call `Backtrack()` on $X_2 = 1$ when X_1 is already set to 1.

Using this call stack, we can produce the list of calls in the order they are explored. For this example where we assign variables in order X_1, X_2, X_3 , the list would be

$\{[01], [01], [01]\}, \{0, [01], [01]\}, \{0, 1, [01]\}, \{0, 1, 0\}, \{1, [01], [01]\}, \{1, 0, [01]\}, \{1, 0, 1\}.$

Suppose we assign variables in the order X_3, X_1, X_2 .

- 1) How many calls do we make to `Backtrack()`? Please provide only the numeric value.
- 2) Why can this number change depending on the ordering? Describe your **explanation** precisely and concisely in one or two sentences.

In keeping with the given example, you should run `Backtrack()` to find all consistent assignments.

Problem 1c [3 Points]

We often add heuristics like AC-3 to speed up the backtracking search.

- 1) How many calls to `Backtrack()` from your call stack in the previous question would we skip if we use AC-3? Please provide only the numeric value.
- 2) Please briefly explain in one or two sentences why we skip (or don't skip) calls in this search when using AC-3.

Problem 2. CSP solving

We'll now pivot towards creating more complicated CSPs, and solving them faster using heuristics. Notice we are already able to solve the CSPs because in `submission.py`, a basic backtracking search is already implemented. For this problem, we will work with *unweighted* CSPs that can only have True/False factors; a factor outputs 1 if a constraint is satisfied and 0 otherwise. The backtracking search operates over partial assignments and specifies whether or not the current assignment satisfies all relevant constraints. When we assign a value to a new variable X_i , we check that all constraints that depend only on X_i and the previously assigned variables are satisfied. The function `satisfies_constraints()` returns whether or not these new factors are satisfied based on the `unaryFactors` and `binaryFactors`. When `satisfies_constraints()` returns False, any full assignment that extends the new partial assignment cannot satisfy all of the constraints, so *there is no need to search further with that new partial assignment*.

Take a look at `BacktrackingSearch.reset_results()` to see the other fields which are set as a result of solving the weighted CSP. You should read `submission.BacktrackingSearch` carefully to make sure that you understand how the backtracking search is working on the CSP.

Problem 2a [5 points]

Let's create an unweighted CSP to solve the n -queens problem: Given an $n \times n$ board, we'd like to place n queens on this board such that no two queens are on the same row, column, or diagonal. Implement `create_nqueens_csp()` by **adding n variables** and some number of binary factors. Note that the solver collects some basic statistics on the performance of the algorithm. You should take advantage of these statistics for debugging and analysis. You should get 92 (optimal) assignments for $n = 8$ with exactly 2057 operations (number of calls to `backtrack()`).

Note: Please implement the domain of variables as 'list' type in Python. You can refer to `create_map_coloring_csp()` and `create_weighted_csp()` in `util.py` as examples of CSP problem implementations. You can try these examples out by running:

```
python run_p1.py
```

Problem 2b [5 points]

You might notice that our search algorithm explores quite a large number of states even for the 8×8 board. Let's see if we can do better. One heuristic we discussed in class is using most constrained variable (MCV): To choose an unassigned variable, pick the X_j that has the fewest number of values a which are consistent with the current partial assignment (a for which `satisfies_constraints()` on $X_j = a$ returns True). Implement this heuristic in `get_unassigned_variable()` under the condition `self.mcv = True`. It should take you exactly 1361 operations to find all optimal assignments for 8 queens CSP—that's 30% fewer! Some useful fields:

- In `BacktrackingSearch`, if `var` has been assigned a value, you can retrieve it using `assignment[var]`. Otherwise `var` is not in `assignment`.

Problem 3: Handling n -ary factors

So far, our CSP solver only handles unary and binary factors, but for course scheduling (and really any non-trivial application), we would like to define factors that involve more than two variables. It would be nice if we could have a general way of reducing n -ary constraints to unary and binary constraints. In this problem, we will do exactly that for two types of n -ary constraints.

Suppose we have boolean variables X_1, X_2, X_3 , where X_i represents whether the i -th course is taken. Suppose we want to enforce the constraint that $Y = X_1 \vee X_2 \vee X_3$, that is, Y is a boolean representing whether at least one variable should be true. For reference, in `util.py`, the function `get_or_variable()` does such a reduction. It takes in a list of variables and a target value, and returns a boolean variable with domain `[True, False]` whose value is constrained to the condition of having at least one of the variables assigned to the target value. For example, we would call `get_or_variable()` with arguments $(X_1, X_2, X_3, \text{True})$, which would return a new (auxiliary) variable X_4 , and then add another constraint $[X_4 = \text{True}]$.

The second type of n -ary factors is constraints on the sum over n variables. You are going to implement reduction of this type.

Problem 3a [5 points]

Suppose we have a CSP with three variables X_1, X_2, X_3 with the same domain $\{0, 1, 2\}$ and a ternary constraint $[X_1 + X_2 + X_3 \leq K]$. How can we reduce this CSP to one with only unary and/or binary constraints? Explain **1)** what **auxiliary variables** we need to introduce, including their **domains**, **2)** what **unary/binary factors** you'll add, providing a brief one sentence of your **explanation** for each variable or factor.

- 1) Auxiliary Variables
- 2) (Unary/Binary) Factors

Problem 3b [5 points]

Let's implement `get_sum_variable()`, which takes in a sequence of non-negative integer-valued variables and returns a variable whose value is constrained to equal the sum of the variables. You will need to access the domains of the variables passed in, which you can assume contain only non-negative integers. The parameter `maxSum` is the maximum sum possible of all the variables. You can use this information to decide the proper domains for your auxiliary variables.

How can this function be useful? Suppose we wanted to enforce the constraint $[X_1 + X_2 + X_3 \leq K]$. We would call `get_sum_variable()` on (X_1, X_2, X_3) to get some auxiliary variable Y , and then add the constraint $[Y \leq K]$. Note: You don't have to implement the $\leq$ constraint for this part.